ENTITY FRAMEWORK CORE — ARCHITECTURAL REVIEW

EF Core Mapping & Strategy Analysis

> This technical document provides formal responses to key architectural inquiries regarding Object-Relational Mapping (ORM) design choices when working with Entity Framework Core in .NET applications.

Question 1: Why Use Code First Rather Than DB First?

In Entity Framework Core, developers can choose between the **Code First** and **Database First** approaches. Selecting Code First is generally preferred for modern, domain-driven software design for several key technical reasons:

1. C# Code as the Single Source of Truth

With Code First, the domain model defined in C# serves as the definitive source of truth. Developers write domain classes and rely on EF Core Migrations to derive and execute corresponding database schema changes. This eliminates manual database scripts and synchronization overhead.

2. Native Version Control & Integration

Schema changes are tracked as C# migration files directly inside the code repository (Git). This ensures schema changes are versioned alongside application logic, facilitating seamless peer code reviews and consistent Continuous Integration / Continuous Deployment (CI/CD) pipelines.

3. Domain-Driven Design Alignment

Code First allows software architects to focus on business logic, entity encapsulation, and object object relationships without being constrained by existing database constructs, normalized table structures, or legacy SQL paradigms.

Feature	Code First Approach	DB First Approach
Primary Source	C# Domain Classes	SQL Database Schema
Schema Updates	Automated via EF Core Migrations	Manual SQL scripts + Scaffold-DbContext
Ideal Use Case	New projects, Domain-Driven Systems	Legacy databases, existing shared DBs

Question 2: Why Use Fluent API Over Data Annotations?

While both **Data Annotations** and **Fluent API** configure EF Core entity mappings, the Fluent API (via `OnModelCreating`) is universally considered superior for enterprise-grade applications.

CLEAN ARCHITECTURE

Separation of Concerns

Data Annotations clutter domain entity classes with database-specific attributes (e.g., `[Table]`, `[Column]`, `[StringLength]`). Fluent API decouples persistence configurations entirely from domain entities, keeping entity classes clean and POCO-compliant (Plain Old CLR Objects).

POWER & FLEXIBILITY

Advanced Configuration Support

Certain advanced mapping rules **cannot** be expressed using Data Annotations alone. Fluent API provides full programmatic control for complex configurations such as:

- Composite Primary Keys and Alternate Keys
- Complex Cascade Delete behaviors and Foreign Key constraints
- Value Generators, Precision/Scale specs (e.g., `decimal(8,2)`)
- Shadow properties, Value Converters, and Table-per-Hierarchy (TPH) rules

MODULARITY

Clean Multi-File Architecture

Fluent API allows configurations to be separated into modular configuration classes using `IEntityTypeConfiguration<T>`. This prevents the `DbContext` from becoming a monolith and improves team collaboration.

Summary Comparison

Criteria	Data Annotations	Fluent API
Entity Cleanliness	Polluted with DB-specific attributes	Clean POCOs, completely decoupled
Expressive Power	Limited to basic attributes & validations	Full power over EF Core mapping engine
Maintenance	Hard to manage in large domain models	Highly scalable, supports modular classes